UNIVERSITÀ DI PISA • ADVANCED SOFTWARE ENGINEERING • A.A. 2026/27

Monster Tamers

Documento di architettura e delle scelte progettuali

Progetto	Gioco di collezione di mostri con backend a microservizi
Versione	1.0 — prototipo completo
Data	15 ottobre 2026
Stack	Quarkus 3.27, Java 21, PostgreSQL 16, nginx, Docker Compose

Il documento descrive l'architettura del sistema, le motivazioni delle scelte tecnologiche e di progetto, il modello dei dati, le regole di gioco implementate, le misure di sicurezza, la strategia di test e il deployment. Il contratto REST dettagliato è in `docs/architettura/api-contract.md`.

Indice

1 Introduzione **1**

1.1 Scopo del documento 1

1.2 Il prodotto in sintesi 1

1.3 Requisiti di progetto 1

1.4 Struttura del repository 2

2 Requisiti funzionali **3**

2.1 User stories 3

2.2 Criteri di accettazione 4

3 Architettura **5**

3.1 Vista d'insieme 5

3.2 Decomposizione in servizi 5

3.3 Dipendenze tra servizi 6

3.4 Stack tecnologico 7

4 Comunicazione e contratto API **8**

4.1 Instradamento nel gateway 8

4.2 Convenzioni comuni 8

4.3 Chiamate interne 8

4.4 Flussi principali 9

4.4.1 Registrazione 9

4.4.2 Avvio e risoluzione di un encounter 9

5 Modello dei dati **11**

5.1 Principi 11

5.2 Entità per servizio 11

5.2.1 auth — `accounts` 11

5.2.2 players — `players` 11

5.2.3 monsters — `monsters`, `abilities`, `owned_monsters` 12

5.2.4 teams — `teams`, `team_members` 12

5.2.5 encounters — `encounter_offers`, `encounter_runs` 12
5.2.6 history — `history_records` 12
5.3 Catalogo 12

6 Logica di gioco **14**
6.1 Crescita dei mostri 14
6.2 Stage e bacheca degli encounter 14
6.3 Motore di battaglia 14
6.4 Ricompense 15
6.5 Bilanciamento 16

7 Sicurezza **17**
7.1 Modello delle minacce 17
7.2 Gestione delle chiavi 18
7.3 Integrità dello storico 18

8 Persistenza e consistenza **19**
8.1 Transazioni locali 19
8.2 Operazioni distribuite 19
8.3 Durabilità 19

9 Frontend **20**
9.1 Scelta tecnologica 20
9.2 Struttura 20
9.3 Separazione delle responsabilità 20

10 Deployment **21**
10.1 Composizione 21
10.2 Build 21
10.3 Configurazione 21
10.4 Avvio 22

11 Test e qualità **23**
11.1 Strategia 23
11.2 Unit test 23
11.3 Test API 23

11.4 Test di prestazione 24

12 Limiti e sviluppi futuri **25**

A Riferimento degli endpoint pubblici **26**

B Variabili di configurazione **27**

CAPITOLO 1

Introduzione

1.1 Scopo del documento

Il documento descrive il sistema *Monster Tamers* realizzato per il progetto di laboratorio di Advanced Software Engineering. Ha due obiettivi: spiegare *come* è costruito il sistema e motivare *perché* è stato costruito così. Per ogni decisione rilevante sono indicate le alternative considerate e il criterio che ha portato alla scelta.

Il documento si rivolge ai membri del gruppo, al docente e a chiunque debba estendere o mantenere il sistema.

1.2 Il prodotto in sintesi

Monster Tamers è un gioco di collezione di mostri. Il giocatore:

1. crea un account e sceglie 3 mostri iniziali da un insieme di 10 estratti per lui;
2. compone dei team di massimo 3 mostri;
3. affronta *encounter* contro mostri selvatici o contro il guardiano dello stage;
4. in caso di vittoria ottiene esperienza, può catturare il mostro sconfitto e, battendo il guardiano, avanza di stage;
5. consulta in ogni momento collezione, catalogo e storico delle battaglie.

1.3 Requisiti di progetto

I requisiti tecnici della consegna e il modo in cui sono soddisfatti sono riassunti in Tabella 1.1.

Requisito	Soluzione
Applicazione distribuita	Sei microservizi indipendenti più un gateway, ognuno in un proprio container
Interfaccia REST	Tutte le funzionalità sono esposte come API REST/JSON, documentate con OpenAPI
Almeno un database	PostgreSQL, con un database e un utente dedicati per ogni servizio
Meccanismi di sicurezza	HTTPS, JWT firmati RS256, bcrypt, rate limiting, controllo di proprietà, catena di hash sullo storico (Capitolo 7)
Unit test e test di prestazione	179 unit test JUnit, 132 test API end-to-end, test di carico Locust (Capitolo 11)
Analisi di sicurezza	Capitolo 7 e `docs/architettura/sicurezza.md`
Frontend minimale senza logica di business	Client web statico che usa solo la REST API
Avvio con container	Un solo comando: `docker compose up -d --build`

Tabella 1.1. Requisiti tecnici della consegna.

1.4 Struttura del repository

Percorso	Contenuto
`backend/<servizio>/`	Un servizio Quarkus per cartella: `pom.xml`, `Dockerfile`, sorgenti, test, README
`backend/monsters/seed/`	Catalogo generato: 100 mostri, 28 abilità, ciclo degli elementi, sprite 64×64
`frontend/`	Client web (HTML, CSS, JavaScript)
`gateway/`	Configurazione nginx e Dockerfile del gateway
`infra/`	Script di inizializzazione di database, chiavi e certificato
`tests/`	Test API (pytest) e test di carico (Locust)
`tools/monster-gen/`	Pipeline di generazione del catalogo e delle immagini
`docs/`	Consegna, design, architettura (questo documento)
`docker-compose.yml`	Definizione dell'intero sistema

CAPITOLO 2

Requisiti funzionali

2.1 User stories

Il backlog contiene 28 user stories. Il colore indica la priorità di progetto: *rossa* per il prototipo, *gialla* per la consegna finale, *verde* opzionale. La Tabella 2.1 riporta per ciascuna il servizio che la realizza.

Tabella 2.1. User stories e servizi responsabili.

ID	User story	Priorità	Realizzata da
US-01	Creare un account	rossa	auth, players
US-02	Login	rossa	auth
US-03	Ritrovare i progressi salvati	rossa	players, PostgreSQL
US-04	Personalizzare nome e avatar	verde	players
US-05	Proteggere i dati dell'account	gialla	auth, gateway, tutti
US-06	Ricevere una collezione iniziale	rossa	players, monsters
US-07	Scegliere il primo mostro	verde	players
US-08	Scegliere una fazione	verde	*non realizzata*
US-09	Vedere la collezione personale	rossa	monsters
US-10	Sfogliare il catalogo	rossa	monsters
US-11	Vedere i dettagli di un mostro	rossa	monsters
US-12	Ottenere un mostro guadagnato	rossa	encounters, monsters
US-13	Sviluppare i mostri	verde	monsters (livelli ed esperienza)
US-14	Creare un team	rossa	teams
US-15	Modificare un team	rossa	teams
US-16	Validare un team	rossa	teams
US-17	Sfogliare gli encounter	rossa	encounters
US-18	Avviare un encounter	rossa	encounters
US-19	Risolvere un encounter	rossa	encounters
US-20	Dettagli del risultato	rossa	encounters
US-21	Abbandonare un encounter	verde	*non applicabile*[a]

ID	User story	Priorità	Realizzata da
US-22	Scegliere le azioni in battaglia	verde	*non realizzata*[a]
US-23	Vedere lo storico	rossa	history
US-24	Dettagli di un encounter passato	rossa	history
US-25	Proteggere i salvataggi	gialla	history, PostgreSQL
US-26	Gestire gli amici	verde	*non realizzata*
US-27	Sfidare un amico	verde	*non realizzata*
US-28	Interfaccia semplice	rossa	frontend

[a] La battaglia è risolta interamente dal server in un'unica richiesta (Sezione 6.3); non esiste uno stato intermedio da cui fuggire o in cui scegliere un'azione.

Tutte le user stories rosse e gialle sono realizzate. Delle verdi sono realizzate US-04, US-07 e US-13, che si integrano senza costi nel modello dei dati scelto.

2.2 Criteri di accettazione

I criteri di accettazione delle storie dell'area giocatori e mostri sono in `docs/design/area_giocatori_mostri.md`. Quelli delle altre aree sono codificati nel contratto API e verificati dalla suite di test end-to-end (Capitolo 11), che riporta per ogni file le user stories coperte.

CAPITOLO 3

Architettura

3.1 Vista d'insieme

Il sistema segue un'architettura a microservizi con un unico punto di ingresso (Figura 3.1). Il browser comunica solo con il gateway tramite HTTPS. Il gateway serve il frontend statico e inoltra le richieste `/api/*` al servizio competente. I servizi comunicano tra loro con chiamate REST sincrone su una rete Docker privata e ciascuno accede soltanto al proprio database.

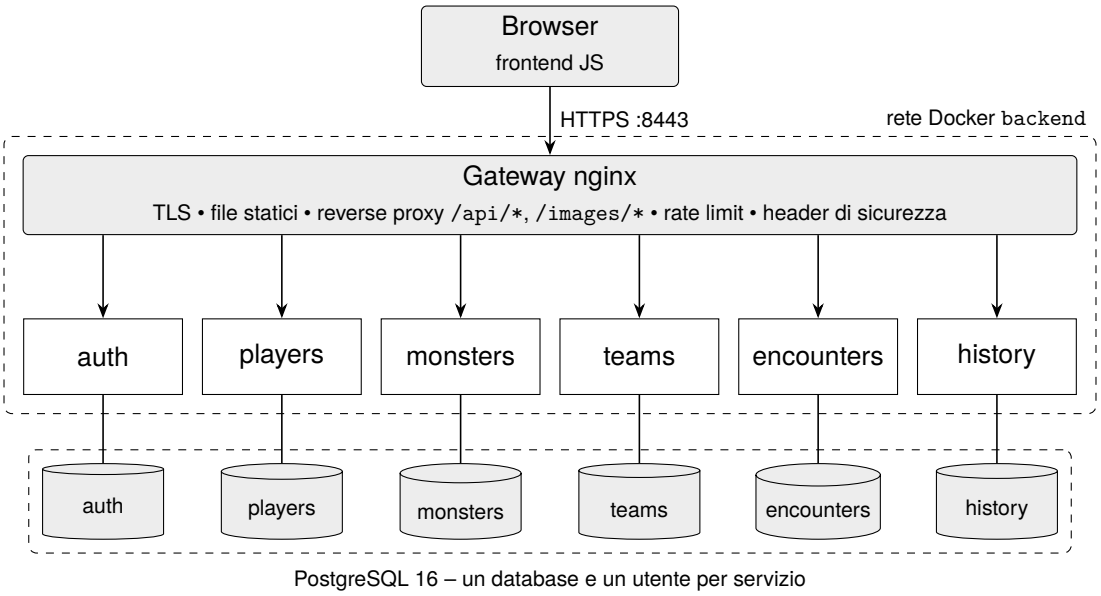


Figura 3.1. Architettura a livello di container.

3.2 Decomposizione in servizi

La suddivisione segue i *bounded context* del dominio: ogni servizio possiede un insieme coeso di entità e ne è l'unico autore. La Tabella 3.1 riassume le responsabilità.

Criteri di decomposizione.

- **Credenziali isolate.** `auth` è l'unico servizio che conosce password e chiave privata di firma. Il profilo di gioco sta in `players`, così i dati sensibili restano confinati in un componente piccolo e facile da verificare.
- **Il catalogo e la collezione insieme.** Le statistiche effettive di un mostro posseduto dipendono dalla specie (statistiche base e crescita). Tenere `OwnedMonster` nello stesso servizio del catalogo evita una chiamata remota per ogni mostro mostrato.

Servizio	Responsabilità	Entità
auth	Registrazione, verifica delle credenziali, emissione dei JWT	`Account`
players	Profilo di gioco, pool e scelta degli starter, avanzamento di stage, progressi aggregati	`Player`
monsters	Catalogo, abilità, elementi, sprite; collezione dei giocatori, statistiche effettive, esperienza e livelli	`Monster`, `Ability`, `OwnedMonster`
teams	Composizione dei team e loro validazione	`Team`
encounters	Bacheca degli encounter, motore di battaglia, calcolo e assegnazione delle ricompense	`EncounterOffer`, `EncounterRun`
history	Storico persistente delle battaglie, catena di hash, statistiche	`HistoryRecord`

Tabella 3.1. Servizi e responsabilità.

- **Logica di gioco in un solo punto.** `encounters` orchestra la battaglia ma non possiede né la collezione né lo stage: chiede ai servizi proprietari di applicare le modifiche. Ogni dato ha un solo autore.
- **Storico separato.** `history` è un archivio in sola aggiunta con propri requisiti di integrità (US-25); separarlo rende la catena di hash indipendente dalla logica di gioco.

Alternative considerate. Un *monolite modulare* sarebbe stato più semplice da sviluppare, ma la consegna richiede un'applicazione distribuita. Una decomposizione più fine (per esempio catalogo e collezione separati) avrebbe moltiplicato le chiamate di rete senza un beneficio reale di indipendenza.

3.3 Dipendenze tra servizi

Le chiamate interne formano un grafo aciclico (Figura 3.2). `monsters` e `history` non chiamano nessuno; `encounters` è l'unico orchestratore.

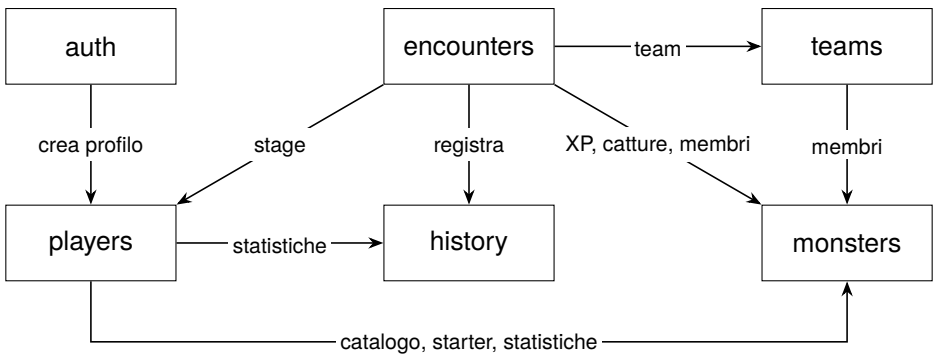


Figura 3.2. Chiamate interne tra servizi (la freccia va dal chiamante al chiamato).

Nessun servizio chiama `auth`: ogni servizio verifica i token localmente con la chiave pubblica. Così `auth` non diventa un collo di bottiglia né un punto di guasto per ogni richiesta.

3.4 Stack tecnologico

Ambito	Scelta	Motivazione
Servizi	Quarkus 3.27 (LTS), Java 21	Avvio rapido e memoria contenuta (rilevanti con sei JVM); estensioni integrate per REST, JPA, JWT, OpenAPI e health check; versione LTS per stabilità
REST	Quarkus REST + Jackson	Modello JAX-RS standard; JSON in `snake_case` configurato una volta per servizio
Persistenza	Hibernate ORM Panache	Entità semplici con query parametrizzate; schema generato dalle entità
Chiamate interne	MicroProfile REST Client	Interfacce dichiarative, header del token interno aggiunto in modo dichiarativo
Database	PostgreSQL 16	Relazionale, transazioni e lock affidabili (necessari per sequenze dello storico e XP)
Gateway	nginx 1.27	Terminazione TLS, reverse proxy, file statici e rate limiting in un solo componente leggero
Frontend	HTML, CSS, JavaScript senza framework	Nessuna build e nessuna dipendenza; il client non contiene logica e resta piccolo (circa 1000 righe)
Deployment	Docker Compose	Un comando avvia l'intero sistema; build multi-stage senza Maven locale
Test	JUnit 5, pytest, Locust	Unit test sulla logica pura; test black-box del contratto; carico realistico

Tabella 3.2. Tecnologie e motivazioni.

Perché un database per servizio. Il pattern *database per service* impedisce a un servizio di leggere o modificare i dati di un altro aggirandone le regole. Per contenere le risorse si usa una sola istanza PostgreSQL con sei database e sei utenti distinti, privi di privilegi reciproci. Separare in seguito le istanze non richiede modifiche al codice, solo alla configurazione.

Perché comunicazione sincrona. Tutte le interazioni sono richiesta–risposta brevi e l'utente attende il risultato (per esempio l'esito di una battaglia). Un broker di messaggi avrebbe aggiunto un componente da gestire senza un caso d'uso che ne tragga beneficio. I limiti di questa scelta sono discussi nella Sezione 8.

CAPITOLO 4

Comunicazione e contratto API

4.1 Instradamento nel gateway

Il gateway applica una regola uniforme: `/api/<servizio>/...` viene inoltrato a `http://<servizio>:8080/<servizio>/...`. Fanno eccezione `/api/collection/...` e `/images/...`, entrambi serviti da `monsters`. I percorsi `/internal/*` e `/q/*` (health check) non sono mai inoltrati: dall'esterno restituiscono 404.

I nomi dei servizi sono risolti dal DNS di Docker a ogni richiesta, quindi il gateway si avvia anche se un servizio non è ancora disponibile e lo raggiunge appena torna in linea.

4.2 Convenzioni comuni

- **Formato.** JSON con nomi in `snake_case`; date in ISO-8601 UTC; identificatori UUID per le entità create a runtime, interi per il catalogo.
- **Errori.** Corpo uniforme `{"error", "message", "details"}` con codici stabili: 400 `validation_error`, 401 `unauthorized`, 403 `forbidden`, 404 `not_found`, 409 `conflict`, 429 `too_many_requests`, 503 `upstream_error`.
- **Autenticazione.** Header `Authorization: Bearer <JWT>`. L'identità del giocatore è sempre il `sub` del token, mai un parametro inviato dal client.
- **Proprietà.** Una risorsa di un altro giocatore restituisce 403; una risorsa inesistente 404.
- **Documentazione.** Ogni servizio pubblica la propria specifica OpenAPI su `/api/<servizio>/openapi` e la Swagger UI su `/api/<servizio>/swagger-ui`.

4.3 Chiamate interne

Le API interne vivono sotto `/internal/...` e sono protette da due livelli indipendenti:

1. il gateway non le inoltra, quindi sono raggiungibili solo dalla rete Docker;
2. ogni servizio richiede l'header `X-Internal-Token`, confrontato in tempo costante con il segreto condiviso configurato.

Un errore o un timeout del servizio chiamato si traduce in 503 per il client. Le chiamate di sola lettura usate per comporre dati accessori (i progressi nel profilo) tollerano il guasto: il campo corrispondente vale `null` e la risposta resta 200.

4.4 Flussi principali

4.4.1 Registrazione

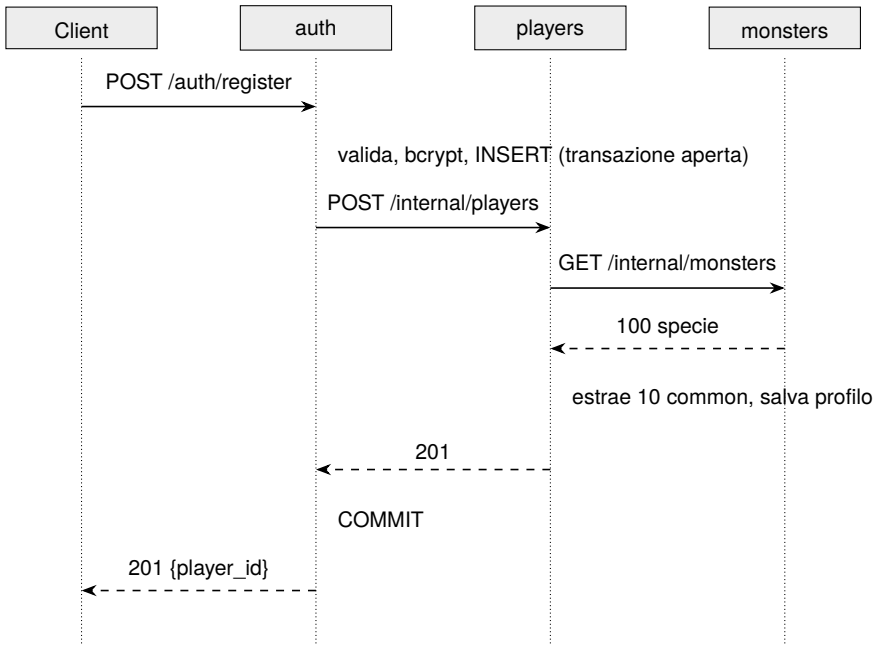


Figura 4.1. Registrazione: se la creazione del profilo fallisce, la transazione dell'account viene annullata.

La creazione del profilo avviene *dentro* la transazione dell'account: se `players` non risponde l'account non viene salvato e il client riceve 503. Così non possono esistere account senza profilo. La creazione del profilo è idempotente, quindi un nuovo tentativo non produce duplicati.

4.4.2 Avvio e risoluzione di un encounter

Il client invia una sola richiesta e riceve l'esito completo, compreso il log degli eventi che il frontend riproduce come animazione. Le richieste dello stesso giocatore sono serializzate per evitare doppie ricompense in caso di doppio invio.

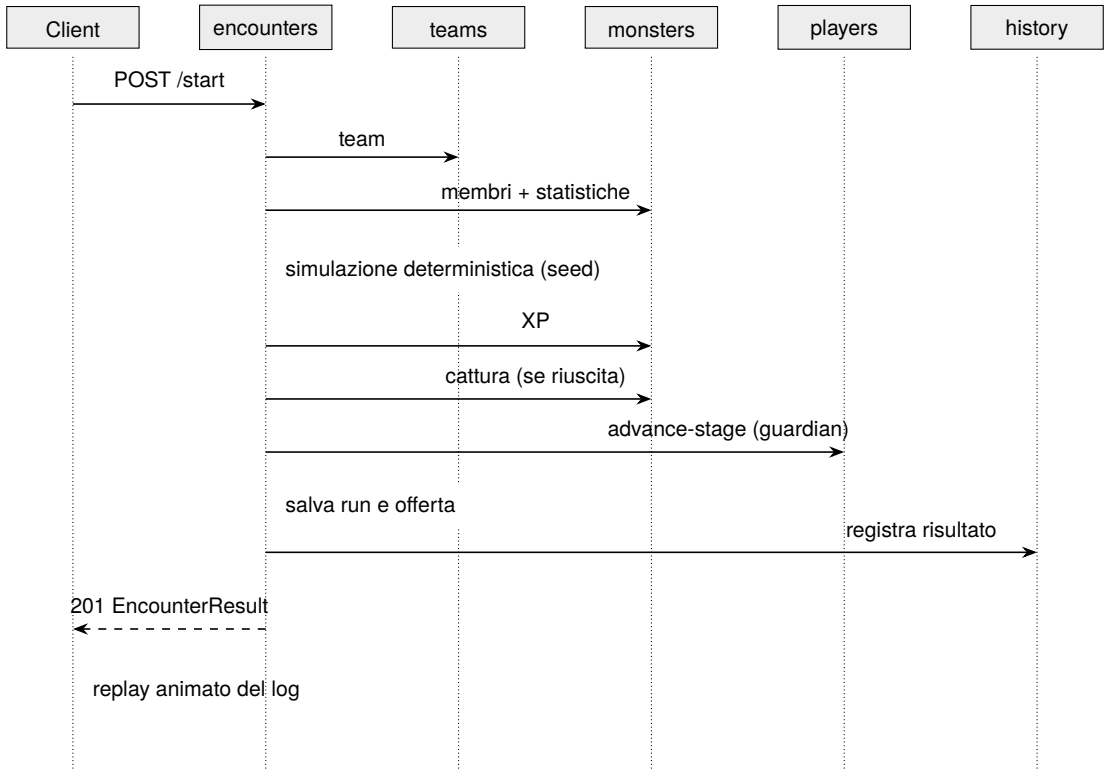


Figura 4.2. Flusso di `POST /encounters/{id}/start`.

CAPITOLO 5

Modello dei dati

5.1 Principi

- Ogni entità appartiene a un solo servizio. I riferimenti tra servizi sono *logici*: si salva l'identificatore (per esempio `player_id`), non una chiave esterna tra database.
- Il `player_id` è lo stesso UUID in tutti i servizi ed è il `sub` del JWT.
- I mostri posseduti sono *istanze* (`owned_id`), non specie: i duplicati sono ammessi e ognuno ha livello ed esperienza propri. Team e battaglie riferiscono istanze.
- I valori derivati non sono salvati: le statistiche effettive si calcolano dal livello, i progressi del giocatore da collezione e storico.

5.2 Entità per servizio

5.2.1 auth — `accounts`

Campo	Tipo	Note
`player_id`	UUID, PK	identità condivisa da tutti i servizi
`username`	testo, unico	3–20 caratteri, salvato in minuscolo
`password_hash`	testo	bcrypt, costo 12, salt incluso
`failed_attempts`, `locked_until`	intero, istante	protezione da brute force
`created_at`, `last_login`	istante	

5.2.2 players — `players`

Campo	Tipo	Note
`player_id`	UUID, PK	
`display_name`	testo	3–20 caratteri, modificabile
`avatar_monster_id`	intero, nullable	deve essere una specie posseduta
`stage`	intero	1–10; 11 indica gioco completato
`starter_pool`	lista ordinata di interi	10 common estratti alla creazione, fissi
`starters_chosen`	booleano	la scelta si fa una sola volta

5.2.3 monsters — monsters, abilities, owned_monsters

Campo	Tipo	Note
Monster (*100 righe di seed*)		
monster_id, name	intero PK, testo	
element, rarity	enum	7 elementi; common / rare / legendary
base_*, growth_*	interi 1–100, decimali	hp, attack, defense, speed
ability_id, image_url	riferimento, testo	
Ability (*28 righe di seed*)		
effect_type, effect_value, trigger	enum, numero, enum	effetto descritto come dato, applicato da encounters
OwnedMonster		
owned_id	UUID, PK	istanza posseduta
player_id, monster_id	riferimenti	
level, xp	interi	livello da 1 a 50
source, encounter_id	enum, UUID nullable	starter o encounter

5.2.4 teams — teams, team_members

Un team ha team_id, player_id, name e una lista *ordinata* di owned_id (tabella team_members con colonna di posizione): l'ordine è l'ordine di ingresso in battaglia.

5.2.5 encounters — encounter_offers, encounter_runs

L'offerta descrive un encounter disponibile (tipo, stage, specie e livello del nemico, stato open/completed). Il run conserva l'esito di una battaglia: seed, risultato completo in JSON e data.

5.2.6 history — history_records

Ogni record contiene il risultato completo della battaglia (payload, testo JSON), il numero di sequenza per giocatore e i campi della catena di integrità prev_hash e hash. I vincoli di unicità su run_id e su (player_id, sequence) impediscono duplicati.

5.3 Catalogo

Il catalogo (100 mostri, 28 abilità, ciclo degli elementi, sprite) è prodotto dalla pipeline tools/monster-gen ed esportato in backend/monsters/seed/. Il servizio lo carica nel

database al primo avvio e lo tiene in memoria, perché non cambia a runtime. Gli sprite sono serviti come file statici e non sono salvati nel database.

Elementi	**Common**	**Rare**	**Legendary**	**Totale**
Chaos	1	4	3	8
Dream, Memory (ciascuno)	9–10	4–5	1–2	16
Time, Echo, Gravity, Silence (ciascuno)	9–10	4–5	1–2	15
Totale	60	30	10	100

CAPITOLO 6

Logica di gioco

Tutte le regole sono implementate nel backend. Il frontend mostra soltanto ciò che le API restituiscono.

6.1 Crescita dei mostri

$$\text{statistica effettiva} = \text{round}\left(\text{base} + \text{growth} \cdot (\text{livello} - 1)\right)$$
$$\text{xp per il livello successivo} = 25 \cdot \text{livello}, \qquad \text{livello massimo} = 50$$

L'esperienza in eccesso si trasferisce al livello successivo, quindi un singolo premio può far salire di più livelli.

6.2 Stage e bacheca degli encounter

Il giocatore progredisce in 10 stage. Per ogni stage la bacheca contiene tre encounter *wild* e un *guardian*.

Aspetto	**Regola (stage s)**
Livello dei wild	casuale in $[3s-2,\ 3s]$; difficoltà easy / normal / hard
Specie dei wild	stage 1–3 solo common; 4–6 common 70% e rare 30%; 7–10 common 30% e rare 70%; mai legendary; specie distinte sulla stessa bacheca
Guardian	livello $3s+2$; uguale per tutti i giocatori; rare negli stage 1–5, legendary negli stage 6–10, elementi a rotazione
Rinnovo	un wild vinto viene sostituito; uno perso resta disponibile
Fine del gioco	battuto il guardian dello stage 10 il gioco è completato; restano disponibili wild di livello stage 10

6.3 Motore di battaglia

La battaglia è simulata dal server in modo *deterministico*: dato il seed, salvato insieme al risultato, la stessa battaglia produce sempre lo stesso log. Questo rende il motore testabile e consente di riprodurre ogni battaglia dallo storico.

- I membri del team combattono in ordine contro il nemico; quando uno va KO entra il successivo.
- In ogni round agisce per primo chi ha velocità maggiore (a parità, il giocatore).
- Vittoria se il nemico va KO; sconfitta se cade tutto il team o dopo 100 round.

$$\text{HP di battaglia} = (2 \cdot \text{hp} + 10) \cdot m, \qquad m = 1\ (\text{team}),\ 1.5\ (\text{wild}),\ 1.4\ (\text{guardian})$$

$$\text{danno} = \max\Big(1,\ \text{round}\Big(\frac{\text{atk}^2}{\text{atk} + \text{def}} \cdot v \cdot e \cdot k\Big)\Big), \quad v \in [0.85, 1]$$

dove e è il moltiplicatore elementale (1.5 se forte, 0.67 se debole, 1 altrimenti) e k raccoglie critico (probabilità 6.25%, $\times 1.5$) ed effetti delle abilità.

Ciclo degli elementi. *Silence* $\rightarrow$ *Echo* $\rightarrow$ *Memory* $\rightarrow$ *Time* $\rightarrow$ *Chaos* $\rightarrow$ *Gravity* $\rightarrow$ *Dream* $\rightarrow$ *Silence*: ogni elemento è forte contro il successivo e debole contro il precedente. Gli elementi sono concettuali per caratterizzare il gioco; la regola a ciclo resta semplice da comprendere e da bilanciare.

Abilità. Ogni abilità è descritta da dati: tipo di effetto, valore e condizione di attivazione. Il motore implementa 13 tipi di effetto e 8 condizioni (Tabella 6.1). Descrivere gli effetti come dati, e non come codice per ogni abilità, permette di aggiungere abilità modificando solo il catalogo.

Effetto	**Semantica (v = `effect_value` in %)**
`damage_boost`	danno inflitto $\times(1+v)$
`damage_reduction`	danno subito $\times(1-v)$
`dodge`	probabilità v di schivare un attacco
`heal`	cura v degli HP massimi (una volta sotto il 50%, oppure a ogni azione)
`speed_boost`	velocità $\times(1+v)$ nel determinare l'ordine
`reflect`	restituisce all'attaccante v del danno subito
`stun`	probabilità v di far saltare il turno all'avversario
`weaken`	attacco dell'avversario $\times(1-v)$ finché resta in campo
`ignore_disadvantage`	in attacco lo svantaggio elementale viene annullato
`revive`	una volta per battaglia, al KO torna con v degli HP
`random_boost`	danno moltiplicato per un valore casuale in $[1-v,\ 1+2v]$
`invert_advantage`	vantaggio e svantaggio elementale scambiati
`copy_ability`	all'ingresso copia l'abilità dell'avversario
Condizioni	`always`, `first_turn`, `hp_above_50`, `hp_below_50`, `on_hit`, `on_attack`, `once`, `vs_faster`

Tabella 6.1. Effetti delle abilità.

6.4 Ricompense

- **Esperienza.** Ogni membro del team riceve $10\times$ livello del nemico (doppio contro il guardian); in caso di sconfitta il 30%.

- **Cattura.** Solo in vittoria: common 60%, rare 35%, guardian sempre. Il mostro catturato entra nella collezione a livello 1, collegato al run che l'ha prodotto.
- **Avanzamento.** La vittoria sul guardian porta allo stage successivo.

Nessun endpoint pubblico permette di aggiungere mostri o esperienza: le ricompense sono assegnate solo da `encounters` tramite le API interne di `monsters` e `players` (US-12).

6.5 Bilanciamento

I parametri sono stati tarati con un test automatico che simula 500 battaglie con i dati reali del catalogo, usando squadre casuali di tre common di livello 1.

Avversario	Vittorie misurate	Intervallo obiettivo
Wild dello stage 1 (common, livello 1–3)	95.8%	75–98%
Guardian dello stage 1 (rare, livello 5)	37.0%	25–80%

Con la prima ipotesi (HP del guardian $\times 2.5$) il guardian dello stage 1 risultava imbattibile (meno dell'1% di vittorie), perché la formula del danno favorisce fortemente chi ha attacco più alto. Si è scelto di correggere i moltiplicatori degli HP, che hanno un effetto lineare e prevedibile, lasciando invariate le formule. Tra le specie candidate a guardian viene scelta quella con attacco base minore, per evitare guardiani con attacco sproporzionato rispetto al livello del giocatore. Il test resta nella suite e segnala ogni modifica che alteri l'equilibrio.

CAPITOLO 7

Sicurezza

7.1 Modello delle minacce

Si considera un attaccante esterno che può raggiungere il gateway e intercettare il traffico di rete, e un giocatore autenticato che tenta di accedere a dati altrui o di ottenere vantaggi non previsti. Le misure adottate sono riassunte in Tabella 7.1.

Tabella 7.1. Minacce e contromisure.

Minaccia	Contromisura
Intercettazione del traffico	Solo HTTPS (TLS 1.2/1.3) terminato dal gateway; la porta HTTP esegue solo un redirect; header HSTS
Furto delle password	Hash bcrypt con costo 12 e salt; password mai salvate né registrate nei log
Attacchi a forza bruta	Blocco dell'account per 5 minuti dopo 5 tentativi falliti; limite di frequenza per IP su `/api/auth/`
Enumerazione degli utenti	Stesso messaggio e stesso tempo di risposta per utente inesistente e password errata (confronto bcrypt eseguito comunque)
Token falsificati	JWT RS256 con scadenza di un'ora e verifica dell'emittente; la chiave privata è montata solo nel container di `auth`
Accesso a dati altrui	Il giocatore è identificato solo dal token; ogni accesso per id verifica la proprietà (403)
Uso diretto delle API interne	Non inoltrate dal gateway; protette da token condiviso confrontato in tempo costante
Manipolazione del gioco	Nessun endpoint per assegnarsi mostri, esperienza o stage; battaglie e premi calcolati solo dal server
Injection	Query parametrizzate; validazione di ogni input (Bean Validation); limiti di dimensione delle richieste
XSS e clickjacking	Content Security Policy restrittiva, `X-Frame-Options: DENY`, `nosniff`; il frontend applica l'escape a ogni valore dinamico
Manomissione dei salvataggi	Catena HMAC sullo storico (Sezione 7.3)
Accesso trasversale ai database	Un utente PostgreSQL per servizio senza privilegi sugli altri database; PostgreSQL non esposto all'esterno
Compromissione di un container	Processi Java eseguiti come utente non privilegiato su immagini minimali

Minaccia	Contromisura
Segreti nel repository	Segreti in `.env` escluso da git; chiavi e certificato generati al primo avvio in volumi Docker

7.2 Gestione delle chiavi

Al primo avvio il container `init-secrets` genera la coppia di chiavi RSA per i JWT e il certificato TLS, e li salva in tre volumi distinti:

Volume	Contenuto	Montato in
`jwt-private`	chiave privata di firma	solo `auth`
`jwt-public`	chiave pubblica di verifica	tutti i servizi
`tls`	certificato e chiave TLS	solo `gateway`

I file esistenti non vengono rigenerati, quindi token e certificato sopravvivono ai riavvii. Ogni servizio verifica i token in autonomia con la chiave pubblica, senza chiamare `auth`.

7.3 Integrità dello storico

Per ogni giocatore lo storico è una catena di record. Il record di sequenza n contiene

$$h_n = \text{HMAC-SHA256}\left(K,\ h_{n-1} \,\|\, n \,\|\, \text{payload}_n\right), \qquad h_0 = \texttt{GENESIS}$$

dove K è un segreto noto solo al servizio `history`. L'endpoint `GET /history/verify` ricalcola la catena e indica il primo record non valido. Una modifica del contenuto, la cancellazione di un record intermedio o un riordino vengono rilevati; senza la chiave non è possibile ricalcolare hash validi.

Le sequenze sono assegnate sotto un lock transazionale per giocatore e protette da un vincolo di unicità, così richieste concorrenti non producono buchi né duplicati nella catena.

CAPITOLO 8

Persistenza e consistenza

8.1 Transazioni locali

Ogni servizio usa transazioni del proprio database. Le operazioni che modificano più righe sono atomiche:

- l'assegnazione degli starter inserisce i tre mostri in un'unica transazione;
- l'assegnazione dell'esperienza è tutto-o-niente: se un mostro non appartiene al giocatore nessuna modifica viene applicata;
- le righe interessate sono bloccate (`PESSIMISTIC_WRITE`) per evitare aggiornamenti persi in caso di richieste simultanee.

8.2 Operazioni distribuite

Un encounter modifica dati di quattro servizi. Il sistema non usa transazioni distribuite né saghe: l'orchestratore esegue i passi in un ordine stabilito (esperienza, cattura, avanzamento, salvataggio del run, storico) e in caso di errore risponde 503.

Scelta consapevole

Il caso peggiore è una ricompensa applicata senza il corrispondente record nello storico. È un'incoerenza a favore del giocatore, non perde dati e non è sfruttabile, perché il client non può provocarla. Una saga con compensazioni avrebbe aggiunto complessità sproporzionata per un prototipo. La Sezione 12 indica come evolvere verso un pattern *outbox*.

Altre misure contro le incoerenze:

- la creazione del profilo e l'avanzamento di stage sono idempotenti;
- `history` rifiuta un secondo record con lo stesso `run_id` (409);
- le richieste di uno stesso giocatore verso `encounters` sono serializzate.

8.3 Durabilità

I dati risiedono nel volume Docker `pgdata` e sopravvivono a riavvii e ricreazioni dei container. Il backup completo si ottiene con `pg_dumpall`.

CAPITOLO 9

Frontend

9.1 Scelta tecnologica

Il frontend è scritto in HTML, CSS e JavaScript con moduli ES nativi, senza framework e senza fase di build. La scelta segue dai vincoli di progetto: il client deve essere minimale e privo di logica di business, quindi i vantaggi di un framework (gestione di stato complesso, componenti riutilizzabili su larga scala) non compensano dipendenze e tooling aggiuntivi. Il risultato sono 14 file per circa 1000 righe, serviti come file statici dal gateway sulla stessa origine delle API (niente CORS).

9.2 Struttura

- `api.js`: chiamate HTTP con il token, gestione uniforme degli errori, logout automatico su 401.
- `router.js`: navigazione basata sull'hash dell'URL e protezione delle viste riservate.
- `ui.js`: template con escape automatico, modali, notifiche, card dei mostri.
- `battle.js`: riproduzione animata del log di battaglia (barre degli HP, danni, critici, efficacia elementale, abilità, cambio di mostro, risultati).
- un modulo per ogni vista: login, starter, home, collezione, catalogo, team, encounter, storico, profilo.

9.3 Separazione delle responsabilità

Il client non calcola nulla che abbia valore di regola. Per esempio:

- la validità di un team è richiesta a `POST /teams/validate` durante la composizione;
- le statistiche mostrate sono quelle effettive calcolate da `monsters`;
- la battaglia non viene simulata nel browser: il client riproduce gli eventi ricevuti.

Ogni comportamento del gioco è quindi verificabile tramite la sola REST API, come richiesto dalla consegna.

CAPITOLO 10

Deployment

10.1 Composizione

Il file `docker-compose.yml` definisce nove container sulla rete privata `backend`:

Container	Immagine	Ruolo
`init-secrets`	alpine 3.20	genera chiavi e certificato, poi termina
`postgres`	postgres 16-alpine	database; crea un database e un utente per servizio
6 servizi	build multi-stage (Maven → JRE 21 alpine)	API; health check su `/q/health/ready`
`gateway`	nginx 1.27-alpine	unico container con porte pubblicate (8080, 8443)

L'ordine di avvio è garantito dalle condizioni di `depends_on`: i servizi partono quando il database è pronto e i segreti sono stati generati; il gateway parte quando tutti i servizi superano l'health check.

10.2 Build

Ogni servizio ha un Dockerfile multi-stage. Il primo stadio compila con Maven ed esegue gli unit test, quindi un'immagine non viene prodotta se i test falliscono. Il secondo stadio contiene solo il runtime Java e l'applicazione e la esegue con un utente non privilegiato. Le dipendenze Maven sono in cache tra una build e l'altra, e non serve Maven installato sulla macchina dello sviluppatore.

10.3 Configurazione

La configurazione è passata tramite variabili d'ambiente con valori di sviluppo predefiniti, in modo che il sistema parta anche senza configurazione. I segreti (password dei database, token interno, chiave dello storico) si impostano nel file `.env`, a partire da `.env.example`.

10.4 Avvio

Comando o indirizzo	Uso
`docker compose up -d --build`	avvia il sistema
`https://localhost:8443`	interfaccia di gioco
`https://localhost:8443/api/<servizio>/swagger-ui`	documentazione interattiva delle API
`docker compose down -v`	arresta e cancella dati, chiavi e certificato

CAPITOLO 11

Test e qualità

11.1 Strategia

I test sono organizzati su tre livelli.

Livello	Strumento	Oggetto
Unit	JUnit 5	Logica di dominio in classi pure, senza database né framework: regole, formule, motore di battaglia, catena di hash
API end-to-end	pytest	Il sistema completo attraverso il gateway, verificando il contratto e le user stories
Prestazioni	Locust	Percorsi realistici di gioco con utenti concorrenti

La logica di dominio è isolata in classi senza dipendenze (per esempio `BattleEngine`, `TeamRules`, `HashChain`, `Growth`, `StarterPolicy`). Questo rende gli unit test veloci, deterministici ed eseguibili a ogni build.

11.2 Unit test

Servizio	Test	Copertura principale
auth	4	blocco dei tentativi, hash con salt
players	33	estrazione e scelta degli starter, avanzamento di stage, validazione del profilo
monsters	48	crescita e livelli, paginazione, filtri, coerenza del catalogo reale
teams	19	regole di composizione dei team
encounters	47	motore di battaglia (tutti gli effetti e le condizioni), bacheca, ricompense, bilanciamento
history	28	catena di hash e rilevamento delle manomissioni, estrazione dei riepiloghi
Totale	**179**	

11.3 Test API

La suite in `tests/api` contiene 132 test che usano il sistema avviato come farebbe un client reale. Ogni test crea i propri utenti, quindi i test sono indipendenti. Oltre ai casi funzionali verifica i requisiti di sicurezza: accesso senza token o con token manomesso, risorse di altri giocatori, endpoint interni non raggiungibili dall'esterno, assenza di endpoint per assegnarsi mostri. L'esecuzione completa dura circa un minuto e risulta interamente superata.

11.4 Test di prestazione

Il test di carico simula giocatori che si registrano, scelgono gli starter, creano un team e poi alternano consultazione del catalogo, della collezione, della bacheca e dello storico con l'avvio di battaglie. I risultati di un'esecuzione con 30 utenti concorrenti per 45 secondi sono riportati di seguito.

Operazione	**Mediana (ms)**	**95° perc. (ms)**	**Errori**
Catalogo (pagine, filtri, ricerca)	3	5	0
Collezione	4	6	0
Bacheca degli encounter	7	14	0
Storico	6	8	0
Avvio e risoluzione di un encounter	86	93	0
Login	230	240	0
Registrazione	270	290	0
Complessivo (842 richieste)	**6**	**230**	**0**

I tempi di login e registrazione sono dominati dal costo di bcrypt, voluto per rendere costoso un attacco alle password. La risoluzione di un encounter include cinque chiamate interne e la simulazione completa.

CAPITOLO 12

Limiti e sviluppi futuri

Limite	Evoluzione possibile
Certificato TLS autofirmato	Certificato emesso da una CA (per esempio Let's Encrypt) o `mkcert` in sviluppo
JWT non revocabili prima della scadenza	Token di accesso brevi con refresh token revocabili
Token interno condiviso tra i servizi	Mutual TLS tra servizi o token di servizio firmati
Operazioni distribuite senza compensazione	Pattern *transactional outbox* per lo storico e le ricompense
Serializzazione degli encounter in memoria	Lock distribuito o vincoli sul database per scalare `encounters` su più istanze
Catena di hash non protegge l'ultimo record	Pubblicazione periodica dell'ultimo hash in un registro esterno
Battaglia non interattiva (US-21, US-22)	Encounter a turni con stato persistito e azioni del giocatore
Funzioni social (US-26, US-27)	Nuovo servizio `social` con amicizie e sfide tra team
Una sola istanza PostgreSQL	Istanze separate per servizio, senza modifiche al codice

APPENDICE A

Riferimento degli endpoint pubblici

Tutti i percorsi sono relativi a `https://localhost:8443/api`. Tutti richiedono un JWT, salvo dove indicato. Il contratto completo, con i formati JSON, è in `docs/architettura/api-contract.md`.

Metodo	Percorso	Descrizione
POST	`/auth/register`	registrazione (senza token)
POST	`/auth/login`	login, restituisce il JWT (senza token)
GET	`/auth/me`	dati dell'account
GET	`/players/me`	profilo, stage, pool degli starter, progressi
PATCH	`/players/me`	nome e avatar
POST	`/players/me/starters`	scelta dei 3 starter
GET	`/monsters`	catalogo paginato con filtri e ricerca
GET	`/monsters/{id}`	dettaglio di una specie
GET	`/monsters/abilities`	elenco delle abilità
GET	`/monsters/elements`	ciclo degli elementi
GET	`/collection`	collezione del giocatore
GET	`/collection/{owned_id}`	dettaglio di un mostro posseduto
GET	`/images/monsters/NNN.png`	sprite (pubblico, fuori da `/api`)
GET	`/teams`	elenco dei team
POST	`/teams`	creazione
GET, PUT, DELETE	`/teams/{team_id}`	lettura, modifica, eliminazione
POST	`/teams/validate`	validazione senza salvataggio
GET	`/encounters`	bacheca dello stage corrente
GET	`/encounters/{id}`	dettaglio di un encounter
POST	`/encounters/{id}/start`	avvio e risoluzione con un team
GET	`/encounters/runs/{run_id}`	risultato di una battaglia
GET	`/history`	storico paginato, filtrabile per esito
GET	`/history/{run_id}`	dettaglio con risultato completo
GET	`/history/verify`	verifica dell'integrità della catena

APPENDICE B

Variabili di configurazione

Variabile	Uso
`POSTGRES_PASSWORD`	password dell'amministratore PostgreSQL
`<SERVIZIO>_DB_PASSWORD`	password dell'utente di database di ciascun servizio
`INTERNAL_TOKEN`	segreto per le chiamate `/internal/*`
`HISTORY_SECRET`	chiave HMAC della catena dello storico
`DB_URL`, `DB_USER`, `DB_PASSWORD`	connessione al database (impostate da Compose)
`JWT_PUBLIC_KEY`, `JWT_PRIVATE_KEY`	percorsi delle chiavi (prefisso `file:`)
`MONSTERS_URL`, `PLAYERS_URL`, `TEAMS_URL`, `HISTORY_URL`	indirizzi dei servizi per le chiamate interne